

Stock Market Analytics — Python ETL

Complete Line-by-Line Explanation of the Actual etl.py Script

This guide explains the exact script provided, from imports and configuration through API extraction, Pandas transformation, MySQL insertion, duplicate prevention, error handling, logging, and the main execution flow.

1. The Big Picture

```
Windows Task Scheduler / Manual Run
|
v
main()
|
+--> create_connection()
|
+--> for each stock
|   |
|   +--> extract_stock_data()
|   |   |
|   |   +--> Alpha Vantage API
|   |   +--> JSON
|   |   +--> DataFrame
|   |
|   +--> insert_new_records()
|   |   |
|   |   +--> MySQL
|
+--> close connection
|
v
END
```

2. Imports — Lines 1–7

import os

Imports Python's operating-system module. Your script uses `os.getenv()` to read configuration values from environment variables.

import time

Provides time-related functions. Your script uses `time.sleep()` to wait between Alpha Vantage requests.

import logging

Provides Python's logging framework. Instead of relying only on `print()`, the ETL records INFO, WARNING, and ERROR messages.

import requests

Third-party HTTP library used to send the GET request to Alpha Vantage.

import pandas as pd

Imports Pandas and gives it the short name `pd`. Pandas is used to create and transform the DataFrame.

import mysql.connector

Imports the MySQL Connector/Python library so Python can connect to and execute SQL against MySQL.

from dotenv import load_dotenv

Imports `load_dotenv()` from `python-dotenv`. It loads variables from your `.env` file into the environment.

3. Configuration

load_dotenv()

This reads your .env file and makes its key/value pairs available through environment variables. The important idea is that secrets such as the API key and database password do not need to be written directly into etl.py.

```
load_dotenv()

API_KEY = os.getenv("ALPHA_VANTAGE_API_KEY")
```

os.getenv() asks the operating system for a variable. If ALPHA_VANTAGE_API_KEY exists, API_KEY receives its value. If it does not exist, API_KEY becomes None.

Database configuration

```
DB_HOST = os.getenv("DB_HOST", "localhost")
DB_USER = os.getenv("DB_USER", "root")
DB_PASSWORD = os.getenv("DB_PASSWORD")
DB_NAME = os.getenv("DB_NAME", "stock_market")
```

The second argument to os.getenv() is a default value. Therefore DB_HOST defaults to localhost, DB_USER defaults to root, and DB_NAME defaults to stock_market if those variables are not present in .env. DB_PASSWORD has no default, so a missing password becomes None and is caught by the validation section.

API URL and stock list

```
API_URL = "https://www.alphavantage.co/query"

STOCKS = [
    "RELIANCE.BSE",
    "TCS.BSE",
    "INFY.BSE",
    "IBM"
]

REQUEST_DELAY = 15
```

API_URL stores the endpoint. STOCKS is a Python list containing the symbols the pipeline will process. REQUEST_DELAY controls how many seconds the program waits between API requests.

4. Logging

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logger = logging.getLogger(__name__)
```

basicConfig() sets the logging configuration. INFO means normal progress messages are shown along with warnings and errors. The format adds the timestamp, severity level, and message. getLogger(__name__) creates a logger associated with the current module.

5. Configuration Validation

```
if not API_KEY:
    raise ValueError(
        "ALPHA_VANTAGE_API_KEY is missing from the .env file."
    )

if not DB_PASSWORD:
    raise ValueError(
        "DB_PASSWORD is missing from the .env file."
    )
```

if not API_KEY checks whether API_KEY is missing/empty. If it is missing, raise ValueError immediately stops the program with a clear message. The same pattern protects the database password. This is better than allowing the script to fail later with a confusing API or database error.

6. extract_stock_data(symbol)

This function is responsible for the **Extract + initial transformation** part of ETL. It receives one stock symbol and returns a Pandas DataFrame, or None if extraction fails.

```
def extract_stock_data(symbol):  
    logger.info(f"Requesting data for {symbol}")
```

def creates a reusable function. symbol is the input parameter. The f-string inserts the actual symbol into the log message.

Building API parameters

```
params = {  
    "function": "TIME_SERIES_DAILY",  
    "symbol": symbol,  
    "outputsize": "compact",  
    "apikey": API_KEY  
}
```

params is a dictionary. requests.get() later converts these key/value pairs into URL query parameters.

TIME_SERIES_DAILY asks for daily time-series data, symbol identifies the stock, compact requests the smaller dataset, and apikey authenticates the request.

Making the HTTP request

```
try:  
    response = requests.get(  
        API_URL,  
        params=params,  
        timeout=30  
    )  
  
    response.raise_for_status()  
  
    data = response.json()
```

try starts an error-handling block. requests.get() sends an HTTP GET request. API_URL is the endpoint and params supplies the query parameters. timeout=30 prevents the program from waiting indefinitely. raise_for_status() raises an exception for HTTP error responses. response.json() converts the JSON response body into a Python dictionary.

Handling request errors

```
except requests.RequestException as e:  
    logger.error(  
        f"HTTP error for {symbol}: {e}"  
    )  
    return None  
  
except ValueError:  
    logger.error(  
        f"Invalid JSON response for {symbol}"  
    )  
    return None
```

RequestException catches network/request-related failures. The variable e contains the exception details. Returning None tells the caller that extraction failed. ValueError handles a failure while parsing JSON.

7. Checking Alpha Vantage's Actual Response

```
if "Time Series (Daily)" not in data:  
  
    if "Note" in data:  
        ...  
  
    elif "Information" in data:  
        ...  
  
    elif "Error Message" in data:  
        ...  
  
    else:  
        ...  
  
    return None
```

This is an important distinction: an HTTP request can succeed while the API still returns an application-level error or rate-limit message. The script therefore checks whether the expected 'Time Series (Daily)' key exists.

Note commonly indicates an API rate-limit message. **Information** can contain API information or access-related messages. **Error Message** indicates a symbol/request error. The final else handles an unexpected response shape. In every case the function returns None.

8. Converting JSON into a DataFrame

```
time_series = data["Time Series (Daily)"]

records = []

for date, values in time_series.items():

    records.append({
        "symbol": symbol,
        "date": date,
        "open": values["1. open"],
        "high": values["2. high"],
        "low": values["3. low"],
        "close": values["4. close"],
        "volume": values["5. volume"]
    })

df = pd.DataFrame(records)
```

data[...] accesses the expected daily series. records starts as an empty list. The for loop walks through each date and its associated price dictionary. Each iteration creates one normal Python dictionary representing one database-ready row. records therefore becomes a list of rows. pd.DataFrame(records) turns that list into a table.

Example conceptual transformation:

API JSON

```
{
  "2026-08-17": {
    "1. open": "3500",
    "2. high": "3550",
    "3. low": "3480",
    "4. close": "3530",
    "5. volume": "120000"
  }
}
```

|
v

DataFrame

symbol	date	open	high	low	close	volume
RELIANCE.BSE	2026-08-17	3500	3550	3480	3530	120000

9. Data Types

```
df["date"] = pd.to_datetime(df["date"]).dt.date

numeric_columns = [
    "open", "high", "low", "close", "volume"
]

for column in numeric_columns:
    df[column] = pd.to_numeric(
        df[column],
        errors="coerce"
    )
```

The API initially provides many values as strings. pd.to_datetime converts the date text into a date/time representation and .dt.date keeps the date portion. numeric_columns stores the columns that should be numeric. The loop converts each of them using pd.to_numeric(). errors='coerce' means a value that cannot be converted becomes NaN rather than crashing the entire pipeline.

10. Removing Invalid Rows

```
df = df.dropna(
    subset=[
        "symbol",
```

```
        "date",  
        "close"  
    ]  
)
```

`dropna()` removes rows where any of the required fields in subset are missing. The script treats symbol, date, and close as essential for a valid stock record. The other numeric fields may still be NaN after conversion, depending on the incoming data.

11. Returning the DataFrame

```
logger.info(  
    f"{symbol}: extracted {len(df)} rows"  
)  
  
return df
```

`len(df)` gives the number of rows after cleaning. `return df` sends the DataFrame back to `main()`, where it will be passed to the MySQL insertion function.

12. MySQL Connection — create_connection()

```
def create_connection():  
    try:  
        connection = mysql.connector.connect(  
            host=DB_HOST,  
            user=DB_USER,  
            password=DB_PASSWORD,  
            database=DB_NAME  
        )  
  
        logger.info(  
            "Connected to MySQL successfully."  
        )  
  
        return connection  
  
    except mysql.connector.Error as e:  
        logger.error(  
            f"MySQL connection failed: {e}"  
        )  
  
        return None
```

`mysql.connector.connect()` creates the connection using the configuration variables. If successful, the connection object is returned. If MySQL raises a connector error, the function logs it and returns `None`. Returning the connection object is important because later functions use it to create cursors and execute SQL.

13. insert_new_records(connection, df)

This function is the **Load** portion of ETL. It receives the existing MySQL connection and the DataFrame for one stock.

```
if df is None or df.empty:  
    logger.warning("No data to insert.")  
    return 0
```

This is a guard clause. If extraction failed and returned `None`, or if the DataFrame has zero rows, there is nothing to load. The function returns 0 because zero records were inserted.

Creating the cursor

```
cursor = connection.cursor()
```

A cursor is the object used to send SQL statements to MySQL and receive execution results.

14. Understanding the INSERT Query

```
INSERT INTO stock_prices  
(  
    symbol, date, open, high, low, close, volume  
)  
SELECT  
    %s, %s, %s, %s, %s, %s, %s  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM stock_prices  
    WHERE symbol = %s  
    AND date = %s  
)
```

The query inserts one row into `stock_prices` only when a row with the same symbol and date does not already exist. The `%s` placeholders are parameter placeholders supplied separately through `cursor.execute()`. This is safer than constructing SQL by concatenating values into the query string.

Why SELECT + WHERE NOT EXISTS? The `SELECT` supplies the values to insert, while `WHERE NOT EXISTS` checks the table first. If the symbol/date pair already exists, the condition is false and no row is inserted.

15. Inserting Each DataFrame Row

```
inserted = 0  
  
for _, row in df.iterrows():  
    values = (  

```

```

        row["symbol"],
        row["date"],
        row["open"],
        row["high"],
        row["low"],
        row["close"],
        int(row["volume"]),
        row["symbol"],
        row["date"]
    )

```

inserted counts successful new rows. iterrows() produces each DataFrame row. The underscore represents the DataFrame index because the index itself is not needed. row is the actual row. values contains nine values because the SQL has seven insertion placeholders plus two placeholders used by the duplicate check. int(row['volume']) converts volume to a Python integer before sending it to MySQL.

16. Executing SQL and Counting Inserts

```

try:
    cursor.execute(
        insert_query,
        values
    )

    if cursor.rowcount > 0:
        inserted += 1

except mysql.connector.Error as e:
    logger.error(
        f"Insert failed for "
        f"{row['symbol']} "
        f"{row['date']}: {e}"
    )

```

cursor.execute() sends the parameterized query and values to MySQL. If MySQL inserted a row, rowcount is greater than zero, so inserted increases by one. If a duplicate prevented insertion, the count does not increase. An error for one row is logged without necessarily stopping the entire loop.

17. Commit and Close

```

connection.commit()

cursor.close()

logger.info(
    f"{inserted} new records inserted."
)

return inserted

```

commit() permanently applies the successful database changes in the current transaction. cursor.close() releases the cursor. The number of newly inserted records is logged and returned to main().

18. main() — The Controller of the ETL

```
def main():  
    logger.info(  
        "===== ETL PROCESS STARTED ====="  
    )  
  
    connection = create_connection()
```

main() orchestrates the complete process. It starts by logging the beginning of the run and then requests a MySQL connection.

Connection failure check

```
if connection is None:  
    logger.error(  
        "Stopping ETL because MySQL connection failed."  
    )  
    return
```

If create_connection() returned None, the pipeline cannot load data. return exits main() cleanly instead of attempting database operations with an invalid connection.

19. Processing Every Stock

```
total_inserted = 0  
  
try:  
    for index, symbol in enumerate(STOCKS):  
  
        logger.info(  
            f"Processing {symbol}"  
        )  
  
        df = extract_stock_data(symbol)  
  
        if df is not None:  
  
            inserted = insert_new_records(  
                connection,  
                df  
            )  
  
            total_inserted += inserted
```

total_inserted keeps a running total for the entire ETL run. enumerate(STOCKS) gives both the position (index) and symbol. For each stock, extraction runs first. If extraction succeeds, the resulting DataFrame is loaded into MySQL. The inserted count is added to the overall total.

20. Why enumerate() is Used

The index is needed later to decide whether a 15-second delay is required. The final stock should not be followed by an unnecessary wait.

```
if index < len(STOCKS) - 1:  
    time.sleep(REQUEST_DELAY)
```

len(STOCKS) is 4. The last index is 3. Therefore the condition is true for indices 0, 1, and 2, but false for 3. This gives waits between requests, but not after the final request.

21. Why time.sleep(15)?

The script deliberately separates requests to reduce the chance of hitting Alpha Vantage's rate limits. The exact allowed limits depend on the API plan and current Alpha Vantage terms, so REQUEST_DELAY should be treated as a configurable safety interval rather than a guaranteed limit.

22. finally — Always Close the Database Connection

```
finally:  
    connection.close()  
  
    logger.info(  
        "===== ETL PROCESS COMPLETED ====="    )
```



```
        "MySQL connection closed."
    )
```

finally executes whether the try block finishes normally or an unexpected exception occurs. This is important because database connections are resources that should be released. It prevents leaving an open connection behind after the ETL run.

23. Final Logging

```
logger.info(
    f"Total new records inserted: {total_inserted}"
)

logger.info(
    "===== ETL PROCESS COMPLETED ====="
)
```

The final messages provide a useful run summary: how many new rows were inserted and that the ETL process reached completion.

24. The `__name__` Check

```
if __name__ == "__main__":
    main()
```

Python assigns `__name__` the value `'__main__'` when this file is executed directly. Therefore `main()` runs when you execute `python etl.py`. If another Python file imports `etl.py`, `__name__` will not be `'__main__'`, so `main()` will not automatically execute. This makes the script reusable as a module.

25. Complete Execution Sequence

```
1. Python starts etl.py
|
2. Imports libraries
|
3. load_dotenv() reads .env
|
4. Configuration variables are loaded
|
5. Logging is configured
|
6. API key and DB password are validated
|
7. __main__ check calls main()
|
8. main() opens MySQL connection
|
9. First stock: RELIANCE.BSE
|
10. API request
|
11. JSON response
|
12. API response validation
|
13. JSON -> records -> DataFrame
|
14. Dates and numeric columns converted
|
15. Invalid required rows removed
|
16. DataFrame sent to insert_new_records()
|
17. Each row checked against MySQL
|
18. Only new symbol/date rows inserted
|
19. Commit transaction
|
20. Wait 15 seconds
|
21. Repeat for TCS.BSE
|
22. Repeat for INFY.BSE
|
23. Repeat for IBM
|
24. Close MySQL connection
|
25. Log total inserted
|
26. ETL completed
```

26. What Happens to One Row?

```
Alpha Vantage
|
| "4. close": "3530.25"
v
Python dictionary
|
| close = "3530.25"
v
Pandas DataFrame
|
| pd.to_numeric()
v
close = 3530.25
|
v
Parameterized SQL
|
| %s <- 3530.25
v
MySQL stock_prices
|
v
Power BI reads analytical data
```

27. What Each Function Does

Function	Purpose	Returns
extract_stock_data(symbol)	Calls Alpha Vantage, validates response, converts JSON to DataFrame, cleans data	DataFrame or None
create_connection()	Creates MySQL connection using environment configuration	Connection or None
insert_new_records(connection, df)	Inserts only records whose symbol/date pair is not already present	Number of inserted rows
main()	Controls the entire ETL workflow	None

28. Why This Script Is a Real ETL Pipeline

Extract: `requests.get()` obtains stock data from Alpha Vantage. **Transform:** the JSON response is converted into a DataFrame, dates/numbers are converted, and invalid required rows are removed. **Load:** `insert_new_records()` sends cleaned records to MySQL. **Reliability:** logging, validation, exception handling, duplicate prevention, transactions, and connection cleanup make the workflow more robust.

29. Important Learning Points

- A Python dictionary is used for API parameters and for representing each output row.
- A list is used for the stock symbols and for collecting row dictionaries before creating the DataFrame.
- Pandas DataFrame is the central tabular structure between extraction and loading.
- `try/except` prevents expected network, JSON, and database errors from crashing the whole pipeline.
- Parameterized SQL uses placeholders instead of manually concatenating values into SQL.
- `WHERE NOT EXISTS` provides application-level duplicate prevention for the symbol/date combination.
- `commit()` makes the database changes permanent.

Official Alpha Vantage documentation: <https://www.alphavantage.co/documentation/>

Python logging documentation: <https://docs.python.org/3/library/logging.html>

Pandas documentation: <https://pandas.pydata.org/docs/>

Requests documentation: <https://requests.readthedocs.io/>

MySQL Connector/Python documentation: <https://dev.mysql.com/doc/connector-python/en/>

python-dotenv documentation: <https://bbc2.github.io/python-dotenv/>